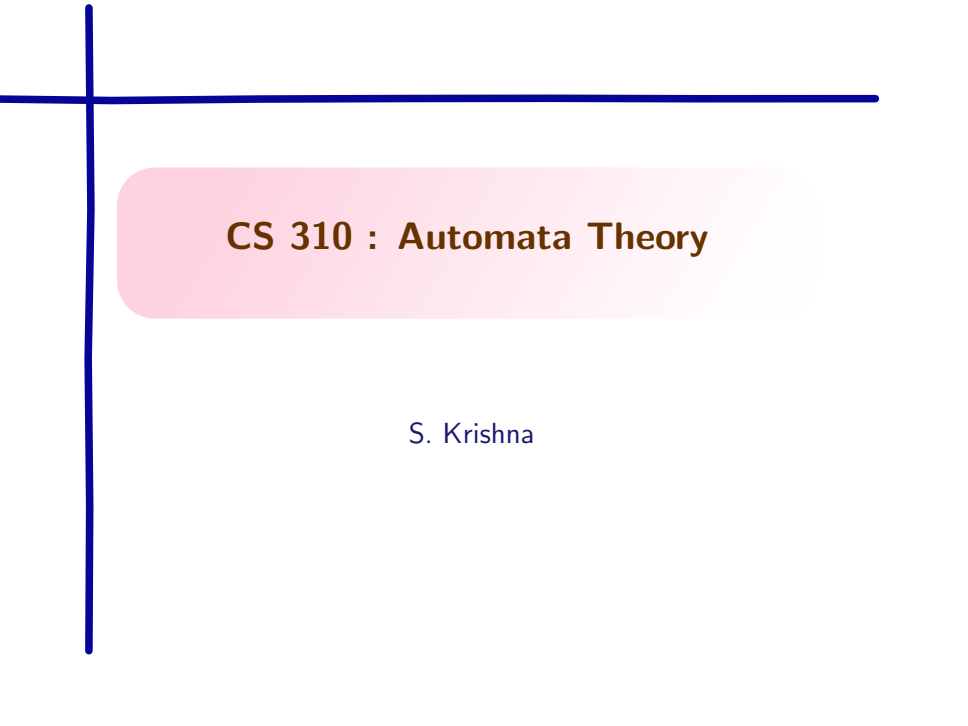


A decorative blue crosshair consisting of a vertical line and a horizontal line intersecting at the top-left corner of the slide.

CS 310 : Automata Theory

S. Krishna

Recap : $\forall\exists$ Automata

Succinct representation over NFAs

- ▶ A $\forall\exists$ automaton is a tuple $(Q, \Sigma, \Delta, S, F)$ where
- ▶ Q is partitioned into $Q_{\forall}$ and $Q_{\exists}$,
- ▶ Transitions out of a $Q_{\exists}$ state are “or” transitions,
- ▶ Transitions out of a $Q_{\forall}$ state are “and” transitions
- ▶ Words have run trees associated with them
- ▶ A word is accepted if it has an accepting run tree
- ▶ An accepting run tree is one whose leaf nodes are accepting states

Are $\forall\exists$ automata more expressive than NFAs?

Is complementation easy for $\forall\exists$ automata?

An Example: Parity Checker

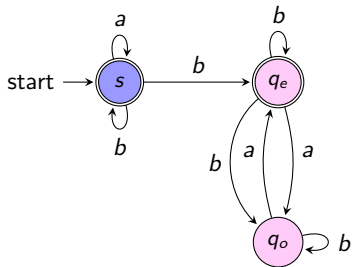
Parity Checker

$L = \{w \in \{a, b\}^* \mid \text{each } b \text{ has an even number of } a\text{'s after it}\}$

An Example: Parity Checker

Parity Checker

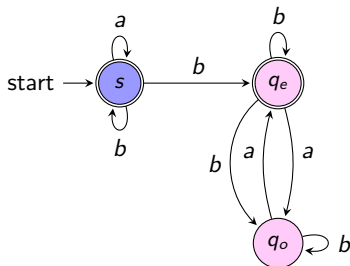
$L = \{w \in \{a, b\}^* \mid \text{each } b \text{ has an even number of } a\text{'s after it}\}$



An Example: Parity Checker

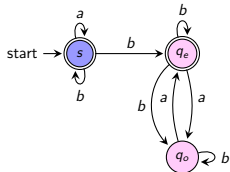
Parity Checker

$L = \{w \in \{a, b\}^* \mid \text{each } b \text{ has an even number of } a\text{'s after it}\}$



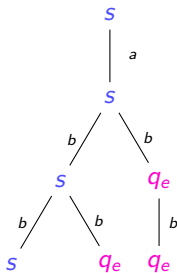
► Blue ($Q_{\forall} = \{s\}$), Pink ($Q_{\exists} = \{q_e, q_o\}$)

Two run trees for $w = abb$

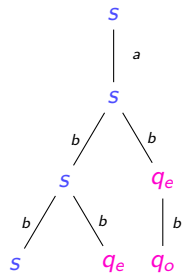


the automaton A , for reference

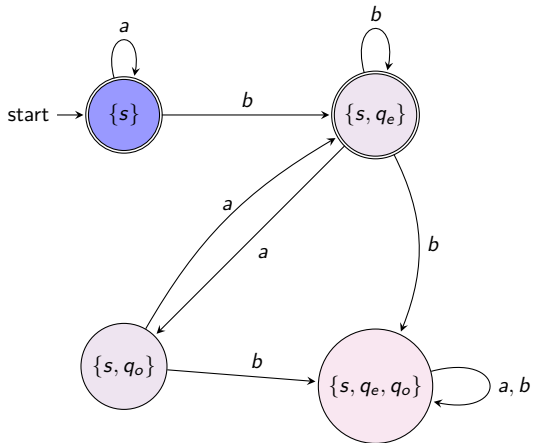
Choice $q_e \rightarrow q_e$



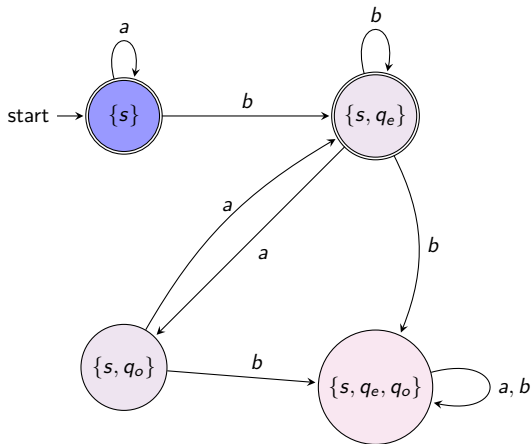
Choice $q_e \rightarrow q_o$



The resulting NFA



The resulting NFA



- ▶ States are reachable subsets of $Q = \{s, q_e, q_o\}$
- ▶ On $\{s, q_e\}$, b has *two* transitions, which stems from q_e 's existential choice.

$\forall\exists$ Automata and NFA accept the same languages

Theorem

Let $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$ be an AFA. Let $A' = (2^Q, \Sigma, \delta', \{s\}, 2^F \setminus \{\emptyset\})$ where,

$$\delta'(X, a) = \left\{ X' \subseteq \delta(X, a) \mid \begin{array}{l} \forall q \in X \cap Q_{\forall}. \delta(q, a) \subseteq X', \\ \forall q \in X \cap Q_{\exists}. X' \cap \delta(q, a) \neq \emptyset \end{array} \right\}.$$

Then $L(A') = L(A)$.

$\forall \exists$ Automata and NFA accept the same languages

Theorem

Let $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$ be an AFA. Let $A' = (2^Q, \Sigma, \delta', \{s\}, 2^F \setminus \{\emptyset\})$ where,

$$\delta'(X, a) = \left\{ X' \subseteq \delta(X, a) \mid \begin{array}{l} \forall q \in X \cap Q_{\forall}. \delta(q, a) \subseteq X', \\ \forall q \in X \cap Q_{\exists}. X' \cap \delta(q, a) \neq \emptyset \end{array} \right\}.$$

Then $L(A') = L(A)$.

► A' is an NFA over states 2^Q

$\forall\exists$ Automata and NFA accept the same languages

Theorem

Let $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$ be an AFA. Let $A' = (2^Q, \Sigma, \delta', \{s\}, 2^F \setminus \{\emptyset\})$ where,

$$\delta'(X, a) = \left\{ X' \subseteq \delta(X, a) \mid \begin{array}{l} \forall q \in X \cap Q_{\forall}. \delta(q, a) \subseteq X', \\ \forall q \in X \cap Q_{\exists}. X' \cap \delta(q, a) \neq \emptyset \end{array} \right\}.$$

Then $L(A') = L(A)$.

- ▶ A' is an NFA over states 2^Q
- ▶ Every $\forall$ -state's *entire* successor set must appear in X' ; every $\exists$ -state needs *one* successor present.

$\forall\exists$ Automata and NFA accept the same languages

Theorem

Let $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$ be an AFA. Let $A' = (2^Q, \Sigma, \delta', \{s\}, 2^F \setminus \{\emptyset\})$ where,

$$\delta'(X, a) = \left\{ X' \subseteq \delta(X, a) \mid \begin{array}{l} \forall q \in X \cap Q_{\forall}. \delta(q, a) \subseteq X', \\ \forall q \in X \cap Q_{\exists}. X' \cap \delta(q, a) \neq \emptyset \end{array} \right\}.$$

Then $L(A') = L(A)$.

- ▶ A' is an NFA over states 2^Q
- ▶ Every $\forall$ -state's *entire* successor set must appear in X' ; every $\exists$ -state needs *one* successor present.
- ▶ Accept iff we can reach a nonempty $X \subseteq F$

Correctness sketch

- ▶ ($\Leftarrow$) If X_i is the set of states at level i of an accepting run tree of A on $a_1 \cdots a_n$, then $X_1 \xrightarrow{a_1} X_2 \xrightarrow{a_2} \cdots \xrightarrow{a_n} X_n$ is an accepting run of A' .

Correctness sketch

- ▶ ($\Leftarrow$) If X_i is the set of states at level i of an accepting run tree of A on $a_1 \cdots a_n$, then $X_1 \xrightarrow{a_1} X_2 \xrightarrow{a_2} \cdots \xrightarrow{a_n} X_n$ is an accepting run of A' .
- ▶ ($\Rightarrow$) If $X \xrightarrow{a} Y$ is a transition of A' , any run tree of A with leaf-labels X extends, reading a , to a run with leaf-labels Y .
- ▶ If $Y \subseteq F$, then we obtain an accepting run tree in A .

Complementation

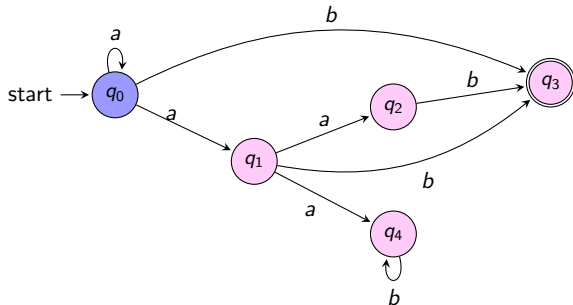
The $\forall\exists$ automaton A

$Q = \{q_0, q_1, q_2, q_3, q_4\}$, $Q_{\forall} = \{q_0\}$, $Q_{\exists} = \{q_1, q_2, q_3, q_4\}$, initial state $\{q_0\}$,
 $F = \{q_3\}$.

Complementation

The $\forall\exists$ automaton A

$Q = \{q_0, q_1, q_2, q_3, q_4\}$, $Q_{\forall} = \{q_0\}$, $Q_{\exists} = \{q_1, q_2, q_3, q_4\}$, initial state $\{q_0\}$,
 $F = \{q_3\}$.



- To show : Each word w accepted by A is rejected by the complement and vice-versa

A two player graph game

Game graph $G(A, w)$

Fix $w = w_1 \cdots w_n \in \Sigma^*$. For *each* such w define game graph $G(A, w)$.

A two player graph game

Game graph $G(A, w)$

Fix $w = w_1 \cdots w_n \in \Sigma^*$. For *each* such w define game graph $G(A, w)$.

- **Vertices** : $Q \times \{1, \dots, n+1\}$ partitioned into two, one for each player

A two player graph game

Game graph $G(A, w)$

Fix $w = w_1 \cdots w_n \in \Sigma^*$. For *each* such w define game graph $G(A, w)$.

- ▶ **Vertices** : $Q \times \{1, \dots, n+1\}$ partitioned into two, one for each player
- ▶ **Edges** : for $a=w_i$, an edge $(q, i) \rightarrow (q', i+1)$ for *every* q' appearing in $\delta(q, a)$.

A two player graph game

Game graph $G(A, w)$

Fix $w = w_1 \cdots w_n \in \Sigma^*$. For *each* such w define game graph $G(A, w)$.

- ▶ **Vertices** : $Q \times \{1, \dots, n+1\}$ partitioned into two, one for each player
- ▶ **Edges** : for $a=w_i$, an edge $(q, i) \rightarrow (q', i+1)$ for *every* q' appearing in $\delta(q, a)$.
- ▶ **Two players** : Adam owns the $\forall$ -states (blue boxes); Eve owns the $\exists$ -states (pink circles).

A two player graph game

Game graph $G(A, w)$

Fix $w = w_1 \cdots w_n \in \Sigma^*$. For *each* such w define game graph $G(A, w)$.

- ▶ **Vertices** : $Q \times \{1, \dots, n+1\}$ partitioned into two, one for each player
- ▶ **Edges** : for $a=w_i$, an edge $(q, i) \rightarrow (q', i+1)$ for *every* q' appearing in $\delta(q, a)$.
- ▶ **Two players** : Adam owns the $\forall$ -states (blue boxes); Eve owns the $\exists$ -states (pink circles).
- ▶ **A move** : a token starts at $(s, 1)$; at (q, i) , the *owner of q* picks one outgoing edge to follow, moving the token to $(q', i+1)$.

A two player graph game

Game graph $G(A, w)$

Fix $w = w_1 \cdots w_n \in \Sigma^*$. For *each* such w define game graph $G(A, w)$.

- ▶ **Vertices** : $Q \times \{1, \dots, n+1\}$ partitioned into two, one for each player
- ▶ **Edges** : for $a=w_i$, an edge $(q, i) \rightarrow (q', i+1)$ for *every* q' appearing in $\delta(q, a)$.
- ▶ **Two players** : Adam owns the $\forall$ -states (blue boxes); Eve owns the $\exists$ -states (pink circles).
- ▶ **A move** : a token starts at $(s, 1)$; at (q, i) , the *owner of q* picks one outgoing edge to follow, moving the token to $(q', i+1)$.
- ▶ **Objective** : Eve wants to **reach** some $(q, n+1)$ with $q \in F$, Adam wants the opposite

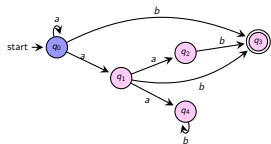
A two player graph game

Game graph $G(A, w)$

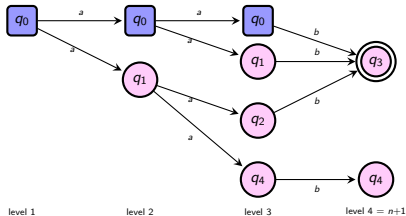
Fix $w = w_1 \cdots w_n \in \Sigma^*$. For *each* such w define game graph $G(A, w)$.

- ▶ **Vertices** : $Q \times \{1, \dots, n+1\}$ partitioned into two, one for each player
- ▶ **Edges** : for $a=w_i$, an edge $(q, i) \rightarrow (q', i+1)$ for *every* q' appearing in $\delta(q, a)$.
- ▶ **Two players** : Adam owns the $\forall$ -states (blue boxes); Eve owns the $\exists$ -states (pink circles).
- ▶ **A move** : a token starts at $(s, 1)$; at (q, i) , the *owner of q* picks one outgoing edge to follow, moving the token to $(q', i+1)$.
- ▶ **Objective** : Eve wants to **reach** some $(q, n+1)$ with $q \in F$, Adam wants the opposite
- ▶ **Winning condition** : after n moves the token is at some $(q, n+1)$: Eve wins iff $q \in F$.

A two player graph game



automaton A



$G(A, w)$ for $w = aab$

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies

A (positional) *strategy* σ_{Eve} (respectively σ_{Adam}) for player Eve (respectively Adam) fixes, at each vertex *that player owns*, which outgoing edge to take.

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies

A (positional) *strategy* σ_{Eve} (respectively σ_{Adam}) for player Eve (respectively Adam) fixes, at each vertex *that player owns*, which outgoing edge to take.

- A play is *consistent with* σ if, at every vertex σ is defined on, the play follows σ 's choice.

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies

A (positional) *strategy* σ_{Eve} (respectively σ_{Adam}) for player Eve (respectively Adam) fixes, at each vertex *that player owns*, which outgoing edge to take.

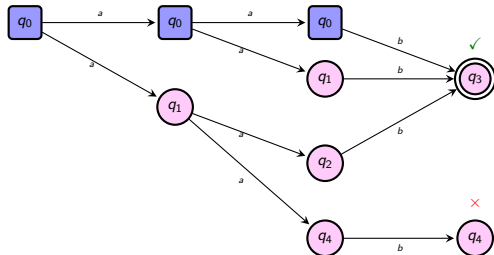
- ▶ A play is *consistent with* σ if, at every vertex σ is defined on, the play follows σ 's choice.
- ▶ σ_{Eve} is a **winning strategy** for Eve if **every** play consistent with σ_{Eve} ends at some $(q, n+1)$ with $q \in F$ irrespective of σ_{Adam} .

To Prove :

Eve has a winning strategy in $G(A, w) \iff A$ accepts w .

Backward reachability, level by level

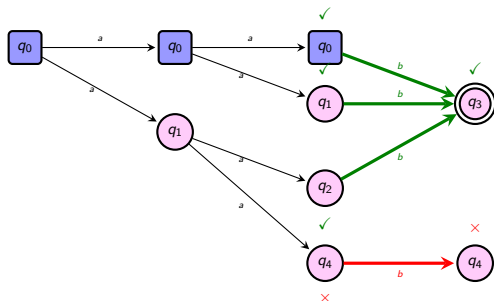
Mark winning states for Eve



- Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.

Backward reachability, level by level

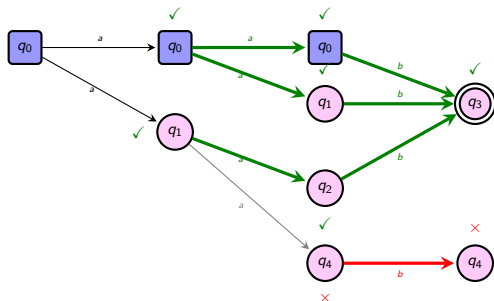
Mark winning states for Eve



- ▶ Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.
- ▶ Level 3: $(q_0, 3)$, $(q_1, 3)$, $(q_2, 3)$ each have one child, q_3 (wins) so they win. $(q_4, 3)$'s only child is $(q_4, 4)$ (loses), it *loses*.

Backward reachability, level by level

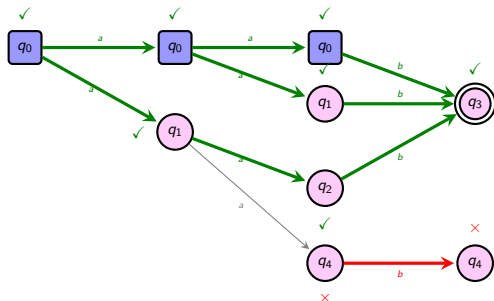
Mark winning states for Eve



- ▶ Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.
- ▶ Level 3: $(q_0, 3)$, $(q_1, 3)$, $(q_2, 3)$ each have one child, q_3 (wins) so they win. $(q_4, 3)$'s only child is $(q_4, 4)$ (loses), it *loses*.
- ▶ Level 2: $(q_0, 2)$ is $\forall$ -owned, has *both* children winning. $(q_1, 2)$ is $\exists$ -owned with *two* choices, q_2 (win) and q_4 (lose), it picks q_2 and *avoids* q_4 : wins.

Backward reachability, level by level

Mark winning states for Eve



- ▶ Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.
- ▶ Level 3: $(q_0, 3), (q_1, 3), (q_2, 3)$ each have one child, q_3 (wins) so they win. $(q_4, 3)$'s only child is $(q_4, 4)$ (loses), it *loses*.
- ▶ Level 2: $(q_0, 2)$ is $\forall$ -owned, has *both* children winning. $(q_1, 2)$ is $\exists$ -owned with *two* choices, q_2 (win) and q_4 (lose), it picks q_2 and *avoids* q_4 : wins.
- ▶ Level 1: $(q_0, 1)$ has *both* children winning $\Rightarrow$ Eve wins from the start $\Rightarrow A$ accepts *aab*.